



**TECNOLÓGICO
DE ESTUDIOS SUPERIORES
DE CUAUTITLÁN IZCALLI**

**T
E
S
C
I**

**“Agente de Segmentación de Audiencia Candidata para Reclutamiento
Automatizado”**

DE RESIDENCIA PROFESIONAL

INFORME TÉCNICO

**INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN Y
COMUNICACIONES**

PRESENTA (AN):

RAMÍREZ MOAR FRANCISCO JAVIER

ASESOR:

DR. JORGE RODRÍGUEZ FIGUEROA

CUAUTITLÁN IZCALLI, EDO. DE MÉXICO 16 ENERO 2026

DECLARATORIA DE AUTORIA LITERARIA ORIGINAL

Declaro que la información que se presenta en este trabajo ha sido investigada, analizada y redactada por **Ramírez Moar Francisco Javier**, habiendo obtenido la información que se refiere en la bibliografía y cuya finalidad es satisfacer los requisitos académicos establecidos para obtener la acreditación de la Residencia Profesional en el campo de Ingeniería en Tecnologías de la Información y Comunicaciones. Además, confirmo que el contenido de este informe técnico es un resultado original, originado en mi trabajo profesional en la compañía Develop Talent & Technology, y que no ha sido copiado ni presentado anteriormente para conseguir otro título o certificación. Asumo la responsabilidad absoluta de la veracidad de los datos y resultados presentados aquí, y le doy permiso al Tecnológico de Estudios Superiores de Cuautitlán Izcalli para que utilice este material únicamente con fines educativos, de consulta y divulgación científica.

AGRADECIMIENTOS

Elaborar este proyecto de residencia profesional no es un logro personal, es el esfuerzo en equipo, el apoyo total y la orientación de personas excepcionales que me acompañarán en el camino. A todos ellos dedico estas palabras.

En primer lugar, quiero expresar mi más profunda y eterna gratitud a mis padres. A ellos, que han sido la raíz sobre el cual he construido mi vida y mis sueños. Gracias por todo el sacrificio realizado para brindarme una educación de calidad, por haber hecho hasta lo imposible para traerme hasta este punto y, sobre todo, por inculcarme los valores de integridad, responsabilidad y perseverancia que hoy son mi brújula profesional. No sería la persona ni el profesional que soy hoy sin la base sólida que ustedes me han dado; este logro es tan suyo como mío.

A mis hermanas y sobrinos, quienes son mi motor y mi fuente inagotable de motivación. Gracias por recordarme, con su presencia y cariño, el porqué de mi esfuerzo y por inspirarme a dar el cien por ciento de mi capacidad todos los días.

A Montserrat Montañez, por brindarme su amistad más sincera y cercana. Gracias por ser mi soporte emocional durante este proceso, por las palabras de aliento en los momentos de incertidumbre y por estar presente incondicionalmente.

Mi sincero reconocimiento a mi asesor y profesor, el Dr. Jorge Rodríguez Figueroa. Gracias por compartir conmigo su sabiduría, no solo técnica sino humana, cualidad única de un verdadero maestro. Agradezco infinitamente su paciencia, su disposición para trabajar conmigo hombro a hombro y su guía constante para superar cada obstáculo que se presentó en el desarrollo de este sistema. Sin su mentoría, la materialización de este proyecto con tal nivel de complejidad habría sido imposible.

De igual manera, extendiendo mi gratitud a todos los profesores que estuvieron presentes apoyándome a lo largo de mi formación académica. Gracias por su vocación, por compartir su

experiencia en las aulas y por brindarme las herramientas teóricas y prácticas que han sido fundamentales para mi desarrollo como ingeniero.

Extiendo mi agradecimiento a la empresa Develop Talent & Technology, y especialmente al Lic. Juan Méndez Herrera, por abrirme las puertas de la organización y confiar en mí para llevar a cabo este proyecto dentro de sus instalaciones, permitiéndome enfrentar retos reales en el mundo laboral.

Al Lic. Ramsés Hernández y al Ing. Ariel Rodríguez González, por su generosidad al compartir sus conocimientos y por brindarme las bases técnicas necesarias para desenvolverme con seguridad en mi trabajo. Gracias por su disposición para resolver mis dudas y por contribuir a mi formación práctica.

Quiero hacer una mención y agradecimiento muy especial a la Ing. María de los Ángeles Flores Guerra González. Gracias por su liderazgo y por guiarme meticulosamente durante todo el proceso, asegurándose de que el trabajo final cumpliera con los más altos estándares de calidad. Agradezco profundamente su paciencia ante mis tropiezos y su capacidad para transformar mis errores en oportunidades de aprendizaje y crecimiento profesional.

Finalmente, a todo el equipo de Recursos Humanos y a mis compañeros de residencia con quienes tuve el placer de colaborar. Ha sido un honor trabajar y aprender al lado de personas talentosas a las que considero futuras grandes promesas del sector.

RESUMEN

Este informe técnico describe con detalle la creación y puesta en marcha de un Agente de Segmentación de Audiencia Candidata, que se incluye dentro de un sistema automatizado de reclutamiento para la compañía Develop Talent & Technology. El propósito principal fue mejorar la etapa de filtrado del talento, que suele ser muy prolongada y estar afectada por prejuicios humanos.

La respuesta fue crear un ecosistema de agentes de inteligencia artificial (IA) que se coordinan a través de un agente orquestador. Como responsable del desarrollo del backend y la lógica de segmentación, se implementó una arquitectura basada en PHP y MySQL, utilizando Gemini CLI para dotar a los agentes de capacidades de razonamiento y procesamiento de lenguaje natural (NLP). El sistema utiliza word embeddings para calcular la similitud semántica entre los perfiles de los candidatos y un modelo de "candidato perfecto" predefinido.

Los hallazgos muestran una mejoría disruptiva en la eficacia operativa: el tiempo para analizar un currículum vitae se acortó a 5 segundos como máximo, alcanzando un grado de exactitud del 80% al identificar candidatos ideales. Asimismo, se aseguró la escalabilidad del sistema y la integridad de los datos a través de una gestión sólida de concurrencia y microservicios. El proyecto no solo automatiza la segmentación, sino que establece una base tecnológica para la evolución hacia el Reclutamiento 4.0, demostrando el potencial de la orquestación de agentes en entornos empresariales reales.

ÍNDICE

INTRODUCCIÓN	1
DESCRIPCIÓN DE LA EMPRESA.....	2
PLANTEAMIENTO DEL PROBLEMA.....	3
OBJETIVOS	4
OBJETIVO GENERAL	4
OBJETIVOS ESPECÍFICOS	4
JUSTIFICACIÓN	4
CAPÍTULO 1. MARCO TEÓRICO	5
1. Fundamentos del reclutamiento automatizado.....	6
Desarrollo y tendencias del proceso de selección y reclutamiento	6
Concepto de selección.....	6
Concepto de reclutamiento	6
Elementos y características	7
Importancia	7
El rol de la inteligencia artificial (IA) en la gestión de talentos	7
Arquitectura del Mini Orquestador de reclutamiento y selección	8
Que es un orquestador.....	8
Clasificación.....	8
2. Inteligencia artificial en RR. HH.	9
Criterios de segmentación de candidatos	9
Como hace esto la IA en programación	9
Modelos de perfilación de puestos (Job Profiling)	9
3. Algoritmos y modelos de aprendizaje en reclutamiento	10
Introducción al aprendizaje no supervisado.....	10
Algoritmos de clustering para segmentación	10

Análisis de componentes principales (PCA).....	10
4. Procesamiento de lenguaje natural (NLP) y preprocesamiento	11
Fuentes de datos y estructura	11
Técnicas de NLP para extracción de características.....	11
Modelos de representación de texto (embeddings)	11
5. Evaluación de desempeño y mantenimiento.....	11
Métricas de evaluación para clustering	11
Validación y optimización	12
Estrategias de integración y monitoreo	12
Arquitectura de Sistemas Multi-Agente y Orquestación en Software.....	12
Sistemas Multi-Agente (SMA) y Automatización Agéntica	12
1.3.2. El Rol del Orquestador en la Integración de Servicios	13
Heurísticas en la Evaluación de Candidatos	13
CAPÍTULO 2. DESARROLLO.....	14
1. Metodología de Desarrollo y Análisis Inicial	15
2. Diseño de la arquitectura del backend	15
3. Diseño e Implementación de la base de datos.....	16
4. Desarrollo de componentes y agentes.....	18
Desarrollo del Agente de Segmentación (ASAC).....	18
Integración de agentes satélite.....	18
5. Integración Frontend-Backend (Vistas Administrativas).....	19
6. Implementación del módulo de autenticación y seguridad	20
7. Desarrollo de la interfaz de postulación y seguimiento	21
8. Integración avanzada de agentes y correcciones del sistema.....	22
Adaptación de la nueva parte del sistema.....	23
Resolución de restricciones de Foreign keys	25
Manejo de restricciones únicas y duplicados	26

Implementación del agente de feedback automatizado.....	27
Mejora del agente de detección de riesgos	28
Corrección del agente de segmentación	29
Integración del Agente de Calendarización con interfaz de usuario	30
Prevención de duplicados en registro de comunicación	31
9. Optimización del flujo de datos entre agentes	32
Consolidación de lógica en el agente orquestador	32
Estrategia de ejecución en cascada para postulaciones.....	32
Registro de logs unificado	33
10. Integración con sistema inicial y mantenimiento de compatibilidad.....	34
Punto central de configuración y compatibilidad bidireccional	34
Mapeo de esquemas y transformación de datos	35
Prevención de conflictos en múltiples ejecuciones	36
CAPÍTULO 3. PROPUESTAS Y RESULTADOS	37
1. Resultados obtenidos	38
Optimización del tiempo de procesamiento.....	38
Precisión y validación del modelo	38
Estabilidad del backend e integración.....	38
2. Mejoras a futuro y propuestas	39
CONCLUSIÓN, RECOMENDACIÓN y EXPERIENCIA PERSONAL PROFESIONAL ADQUIRIDA	40
COMPETENCIAS DESARROLLADAS Y/O APLICADAS	41
REFERENCIAS BIBLIOGRÁFICAS	42

ÍNDICE DE FIGURAS

Imagen 1. Lógica de control centralizada en la clase AgenteOrquestador de Autoría propia (2025).	14
Imagen 2. Estructura de la tabla de auditoría log_agentes de Autoría propia(2025).	15
Imagen 3. Registros de evaluaciones con clasificación automática de "Fit" del candidato de auditoria propia (2025).	15
Imagen 4. Implementación de la clase para el Agente de Validación de Procesos de auditoría propia (2025).	17
Imagen 5 Vista administrativa del módulo de Onboarding con estados actualizados en tiempo real de Autoría propia (2025).	18
Imagen 6 Controlador de inicio de sesión y validación de usuarios de Autoría propia (2025).	19
Imagen 7 Interfaz de usuario para la visualización de vacantes y seguimiento de estado de Autoría Propia (2025).	20
Imagen 8. Archivo config.php modificado de Autoria propia (2025)	21
Imagen 9 fragmento de código helpers_db.php de Autoría propia (2025)	22
Imagen 10. Panel de programación de entrevistas funcionando correctamente tras la resolución de restricciones de foreign keys de Autoría propia (2025).	23
Imagen 11. Sistema de entrevistas funcionando sin errores de duplicados tras implementar la verificación previa de Autoría propia (2025).	24
Imagen 12 base de datos mostrando el proceso automático de feedback al rechazar un candidato de Autoría propia (2025).	25
Imagen 13 Panel de evaluación mostrando riesgos detectados automáticamente por el agente con indicadores de severidad de Autoría	26

propia (2025).	
Imagen 14. Panel de evaluación mostrando los scores calculados automáticamente por el agente de segmentación de Autoría propia (2025).	27
Imagen 15. Interfaz del agente de calendarización mostrando propuestas de horarios y opción de rechazo de Autoría propia (2025).	28
Imagen 16 fragmento de código donde se conecta al canal de comunicación Autoría propia (2025)	28
Imagen 17 Panel de evaluación mostrando los resultados de la cascada de agentes tras una postulación de Autoría propia (2025).	30
Imagen 18 Interfaz de logs mostrando el registro unificado de ejecuciones de agentes de Autoría propia (2025).	30

ÍNDICE DE TABLAS

Figura 1. Elementos y Características de Reclutamiento y selección	5
Figura 2 Mapeo de tabla candidatos	43
Figura 3 mapeo de tabla vacantes	43

LISTA DE ABREVIATURAS Y TABLA DE SÍMBOLOS

Abreviaturas

1. API: Interfaz de Programación de Aplicaciones (Application Programming Interface).
2. ASAC: Agente de Segmentación de Audiencia Candidata.
3. ATS: Sistema de Seguimiento de Candidatos (Applicant Tracking System).
4. BERT: Representaciones de Codificador Bidireccional de Transformadores (Bidirectional Encoder Representations from Transformers).
5. CLI: Interfaz de Línea de Comandos (Command Line Interface).
6. CV: Currículum Vitae.
7. DBSCAN: Agrupamiento Espacial Basado en Densidad de Aplicaciones con Ruido (Density-Based Spatial Clustering of Applications with Noise).
8. DOCX: Formato de archivo de documento extensible de Office (Office Open XML).
9. GPT: Transformador Generativo Preentrenado (Generative Pre-trained Transformer)
- 10.IA: Inteligencia Artificial.
- 11.JSON: Notación de Objetos de JavaScript (JavaScript Object Notation)
- 12.NLP: Procesamiento de Lenguaje Natural (Natural Language Processing).
- 13.PCA: Análisis de Componentes Principales (Principal Component Analysis).
- 14.PDF: Formato de Documento Portátil (Portable Document Format).
- 15.PHP: Preprocesador de Hipertexto (PHP: Hypertext Preprocessor).
- 16.RR. HH.: Recursos Humanos.
- 17.SQL: Lenguaje de Consulta Estructurado (Structured Query Language).

18.TI: Tecnologías de la Información.

19.IA: Inteligencia Artificial

20.UI / UX: Interfaz de Usuario/Experiencia de Usuario

21.HTTP: Protocolo de Transferencia de Hipertexto

Símbolos

Símbolo	Significado
k	Variable que representa el número de grupos o clústeres predefinidos en el algoritmo K-Means.
1	Valor binario en el vector de características que indica la presencia de una habilidad técnica (<i>Hard Skill</i>).
0	Valor binario en el vector de características que indica la ausencia de una habilidad técnica (<i>Hard Skill</i>).



TECNOLÓGICO DE ESTUDIOS SUPERIORES DE CUAUTITLÁN IZCALLI

TESCI

INTRODUCCIÓN

El proceso de reclutamiento y selección de personal representa un desafío operativo y estratégico para las organizaciones modernas. La revisión manual y exhaustiva de volúmenes crecientes de solicitudes genera cuellos de botella, incrementa los costos y es susceptible a sesgos cognitivos. En este contexto, la implementación de tecnologías de la información y la comunicación (TIC), específicamente la Inteligencia Artificial (IA), emerge como una solución imperativa para optimizar y automatizar esta función crítica.

La evaluación subjetiva y la falta de estandarización en el filtrado inicial representan un riesgo significativo en términos de pérdida de talento cualificado e ineficiencia operativa. Por ello, durante el periodo de residencia profesional se planteó el desarrollo de un Agente de Segmentación que actúe como un "filtro inteligente", permitiendo consolidar, evaluar y clasificar automáticamente las postulaciones dentro del ecosistema RecursosH, alineándose con las tendencias de automatización del Reclutamiento 4.0.

Este documento presenta el desarrollo técnico del proyecto, desde la definición de la identidad del agente en XML y su arquitectura lógica, hasta su implementación en un sistema orquestador y la validación final. Asimismo, se detallan los beneficios esperados en términos de agilidad, objetividad algorítmica y trazabilidad de los datos, contribuyendo al fortalecimiento y modernización de la infraestructura tecnológica del departamento de Recursos Humanos.

DESCRIPCIÓN DE LA EMPRESA

Develop Talent & Technology es una organización con 19 años de trayectoria (fundada en 2006) especializada en la formación de capital humano y el desarrollo de soluciones en el sector de Tecnologías de la Información (TI). Su actividad principal se centra en la Capacitación Tecnológica Especializada Integral, operando bajo un modelo de Blended E-Learning que vincula a la academia con las demandas reales del mercado laboral nacional e internacional.

La empresa se destaca por su capacidad de integración entre el sector educativo, la iniciativa privada y el gobierno, logrando los siguientes hitos operativos:

- Alcance y cobertura: Presencia en las 32 entidades federativas de México, colaborando con más de 500 instituciones educativas.
- Certificación de talento: Ha certificado a más de 35,000 alumnos con una tasa de éxito de 9 de cada 10 participantes, gracias a alianzas estratégicas con líderes tecnológicos (Oracle, Microsoft, Google, entre otros) y centros evaluadores (Pearson VUE, Prometric).
- Desarrollo de software: Adicional a la formación, Develop cuenta con una sólida división de desarrollo de sistemas financieros para la banca y casas de bolsa, así como servicios de consultoría e implementación de procesos de TI.

Su misión es democratizar el acceso a certificaciones internacionales de alta validez, cerrando la brecha de habilidades técnicas y potenciando la empleabilidad en la industria del software.

PLANTEAMIENTO DEL PROBLEMA

En el contexto actual de la gestión de capital humano, el departamento de Recursos Humanos de la organización "TODO Academy" enfrenta un desafío operativo crítico derivado de la digitalización de sus canales de atracción de talento. El proceso de reclutamiento vigente depende fundamentalmente de una revisión manual y exhaustiva de currículums (CVs). Al publicar una vacante, los reclutadores reciben volúmenes masivos de postulaciones con datos no estructurados, lo que satura la capacidad de procesamiento del equipo.

Esta dependencia de métodos manuales para la fase de preselección ha generado tres problemáticas sustanciales que afectan la competitividad de la organización:

- Ineficiencia operativa y saturación de recursos: La tarea de filtrado inicial o "criba curricular" consume hasta el 60% del tiempo efectivo del reclutador. Este uso intensivo de recursos humanos en tareas repetitivas y de bajo valor estratégico impide que el personal se enfoque en etapas cruciales como la entrevista y la evaluación cultural.
- Sesgo cognitivo y subjetividad: La selección manual está inherentemente sujeta a sesgos humanos inconscientes. La falta de criterios estandarizados y automatizados puede derivar en el descarte injustificado de talento valioso o en la priorización de perfiles basada en factores subjetivos, comprometiendo la calidad y objetividad de la contratación.
- Cuellos de botella e incremento del time-to-hire: Existe una desconexión operativa entre la recepción de documentos y la gestión de la agenda de entrevistas. Esta falta de integración genera retrasos significativos en el tiempo de contratación (Time-to-Hire), provocando la pérdida de candidatos altamente competitivos frente a otras empresas con procesos más ágiles.

OBJETIVOS

OBJETIVO GENERAL

Desarrollar un agente de IA capaz de segmentar de manera automática y precisa la audiencia de candidatos potenciales, optimizando el proceso de reclutamiento dentro del Mini Orquestador de Reclutamiento y Selección.

OBJETIVOS ESPECÍFICOS

1. Implementar algoritmos de análisis y filtrado que permitan identificar y agrupar candidatos según criterios definidos por el área de recursos humanos.
2. Integrar el agente con fuentes de datos internas y externas para asegurar la actualización y diversidad de la base de candidatos segmentados.

JUSTIFICACIÓN

La segmentación eficiente de candidatos es fundamental para acelerar y mejorar la calidad del proceso de reclutamiento en las organizaciones. Este agente aporta valor al reducir la carga operativa de los reclutadores, permitiendo un enfoque más estratégico en la selección y evaluación. Para TODO Academy, representa un componente clave en la automatización de procesos profesionales, alineado con los estándares de innovación y eficiencia del programa Lead Orchestrator. Para el alumno, el desarrollo de este agente fortalece competencias en inteligencia artificial aplicada, manejo de datos y automatización de procesos de recursos humanos, habilidades demandadas en el mercado laboral actual.

CAPÍTULO 1.

MARCO TEÓRICO

1. Fundamentos del reclutamiento automatizado

La transformación digital ha impulsado a los departamentos de Recursos Humanos a evolucionar desde la gestión operativa manual hacia modelos estratégicos basados en datos. Los fundamentos del reclutamiento automatizado no solo implican la digitalización de currículums, sino la implementación de arquitecturas de software capaces de orquestar flujos de trabajo complejos sin intervención humana constante. Esta sección analiza cómo las tecnologías de automatización lógica y los sistemas de agentes inteligentes permiten reducir la carga administrativa, mitigando sesgos cognitivos y optimizando la precisión en el filtrado de talento mediante reglas heurísticas y algoritmos de procesamiento de información.

Desarrollo y tendencias del proceso de selección y reclutamiento

El proceso de transición del reclutamiento tradicional al digital ha pasado por varias etapas, culminando en el "Reclutamiento 4.0", que se fundamenta en datos.

- Reclutamiento 1.0 y 2.0: Se enfocaban en la divulgación unidireccional (en los medios impresos) y bidireccional (por medio de redes sociales y portales de trabajo), respectivamente.
- Contratación fundamentada en datos: Actualmente, las organizaciones emplean el Big Data para prever el éxito de una contratación.

Concepto de selección

La selección es la etapa posterior al reclutamiento que se enfoca en evaluar a los candidatos para elegir al individuo más adecuado que cumpla con los requisitos del puesto. Es un proceso de diferenciación restrictiva cuyo objetivo es reducir el grupo de aspirantes al mejor candidato (iSmartRecruit, 2025).

Concepto de reclutamiento

El reclutamiento se define como un conjunto de procedimientos orientados a atraer candidatos potencialmente cualificados y capaces de ocupar cargos dentro de la organización. Es fundamentalmente un sistema de información y comunicación: la organización divulga y ofrece oportunidades de empleo al mercado de recursos humanos.

Según autores clásicos como Chiavenato, es un proceso "positivo" de invitación, cuya función es abastecer el proceso de selección con materia prima (candidatos) (Urbe, 2009). A diferencia de la selección, no busca filtrar, sino maximizar la cantidad de talento disponible para el análisis posterior.

Elementos y características

Elemento	Reclutamiento	Selección
Naturaleza	Proceso de atracción (positivo).	Proceso de filtrado (restrictivo).
Objetivo	Suministrar el mayor número de candidatos aptos.	Elegir al mejor candidato disponible (mejor ajuste).
Alcance	Temporalmente termina cuando se reciben las postulaciones.	Se inicia después y termina con la contratación.

Figura 1. Elementos y Características de Reclutamiento y selección

Importancia

La eficacia en el reclutamiento y la selección se traduce en beneficios directos para la organización. Un proceso estratégico, potenciado por tecnología, asegura una mejora de la productividad, la rentabilidad y la competitividad de la empresa (Deel, 2025). La automatización permite la reducción de costos y el uso de tecnología aplicada para agilizar y hacer más objetiva la selección de currículums.

El rol de la inteligencia artificial (IA) en la gestión de talentos

En el contexto de Recursos Humanos, la IA no sustituye al humano, sino que aumenta sus capacidades ("Augmented Intelligence").

- Definición de "Agente de IA": Un Agente de IA es un sistema de software impulsado por inteligencia artificial que interactúa de forma autónoma con su entorno, recopila datos, toma decisiones y realiza tareas complejas para lograr un objetivo determinado. A diferencia de los sistemas tradicionales, un agente de IA es capaz de planificar la división de objetivos amplios en subtareas y mejorar su rendimiento con el tiempo (ServiceNow, n.d.).

- Aplicación: Estos agentes posibilitan la transición de procesos reactivos a otros proactivos, al detectar patrones de talento que un reclutador humano podría no percibir por agotamiento cognitivo.

Arquitectura del Mini Orquestador de reclutamiento y selección

Que es un orquestador

Ambos procesos son cruciales, pero su automatización es vital para la competitividad. Un reclutamiento eficaz asegura una base de talentos amplia y diversa, mientras que una selección eficiente reduce el riesgo de errores costosos (mala contratación) y mejora la retención del personal (Byars y Rue, 2006). La IA busca la sinergia, asegurando que solo los perfiles correctos (atracción) sean evaluados a fondo (decisión).

Clasificación

Los orquestadores pueden clasificarse según su enfoque:

- Orquestación Centralizada: Una entidad única (el Orquestador) conoce y controla el flujo completo de todos los agentes y servicios. Es ideal para procesos bien definidos como tu sistema de reclutamiento.
- Coreografía Descentralizada: Los servicios individuales se comunican directamente entre sí sin un controlador central, reaccionando a eventos. Esto es más flexible pero más difícil de monitorear.
- Ubicación del agente: En la arquitectura híbrida de microservicios del "Mini Orquestador", el Agente de Segmentación de Audiencia Candidata (ASAC) tiene un papel fundamental como nodo intermedio.
- Función sistémica: Se ubica después del "Agente de Extracción de Datos" (que convierte CVs en JSON) y antes del "Agente de Calendarización". Su función es filtrar el ruido (candidatos no aptos) para que los procesos subsiguientes solo consuman recursos en candidatos viables. Funciona bajo un patrón de "publicador-suscriptor", consumiendo el perfil del puesto y emitiendo listas segmentadas.
- Ubicación del agente: En la arquitectura híbrida de microservicios del "Mini Orquestador", el Agente de Segmentación (ASAC) tiene un papel fundamental como nodo intermedio.

El Agente de Segmentación de Audiencia Candidata (ASAC) es el componente lógico central que transforma los datos brutos en información de valor para la toma de decisiones. Su posición dentro de la arquitectura es estrictamente intermedia y secuencial, funcionando bajo un patrón de Procesamiento en Cadena (Pipeline).

- **Función sistémica:** Se ubica después del "Agente de Extracción de Datos" (que convierte CVs en JSON) y antes del "Agente de Calendarización". Su función es filtrar el ruido (candidatos no aptos) para que los procesos subsiguientes solo consuman recursos en candidatos viables. Funciona bajo un patrón de "publicador-suscriptor", consumiendo el perfil del puesto y emitiendo listas segmentadas.

2. Inteligencia artificial en RR. HH.

Criterios de segmentación de candidatos

La segmentación posibilita la división de una base de datos heterogénea en clústeres homogéneos, con el fin de implementar tácticas distintas.

- **Demográficas y geográficas:** Ubicación, disponibilidad de reubicación.
- **Psicográficas:** Valores, cultura laboral y motivaciones (alineación con la misión de la empresa).
- **Hard skills (habilidades técnicas):** Habilidades que pueden ser verificadas (por ejemplo, dominio de SQL, PHP skills (habilidades técnicas)): Competencias verificables (ej. dominio de PHP, SQL).
- **Habilidades blandas (soft skills):** Habilidades interpersonales deducidas del examen semántico del currículum vitae (por ejemplo, liderazgo, comunicación).

Como hace esto la IA en programación

En la programación del agente, la IA convierte estas características en vectores numéricos a través de modelos de Word Embeddings. Por ejemplo, las hard skills son mapeadas a tokens binarios (1 si posee la skill, 0 si no), mientras que las soft skills y valores culturales se evalúan mediante el análisis de similitud coseno entre el vector del CV del candidato y el vector de la descripción del puesto, calculando qué tan cerca están conceptualmente en el espacio vectorial.

Modelos de perfilación de puestos (Job Profiling)

El Job Profiling es la técnica para definir el "Candidato Ideal" (Avatar). En tu sistema, esto se logra mediante la estructuración de la vacante en formato JSON.

- **Correlación:** La segmentación solo es efectiva si se correlaciona con un perfil de puesto bien definido. Si el perfil del puesto es ambiguo, el algoritmo de segmentación generará "falsos positivos" (candidatos que parecen aptos pero no lo son).
- **Sesgos algorítmicos:** Existe el riesgo de que el agente replique sesgos históricos (ej. penalizar ciertos códigos postales o géneros). Es crucial auditar los datasets de entrenamiento para asegurar la equidad (fairness) y cumplir con normativas de privacidad de datos.

3. Algoritmos y modelos de aprendizaje en reclutamiento

Introducción al aprendizaje no supervisado

El aprendizaje no supervisado, en contraste con el aprendizaje supervisado (que necesita que los datos sean etiquetados de antemano como "contratado" o "rechazado"), identifica patrones escondidos en los datos sin necesidad de intervención humana.

Justificación: En reclutamiento, a menudo no sabemos a priori qué tipos de candidatos existen en una base de datos masiva. Los modelos no supervisados permiten descubrir "arquetipos" de candidatos de forma orgánica.

Algoritmos de clustering para segmentación

K-Means: Algoritmo que agrupa candidatos en K grupos basándose en la similitud de sus características (ej. años de experiencia vs. número de habilidades técnicas). Es eficiente computacionalmente, pero sensible a valores atípicos (outliers).

DBSCAN: Útil para identificar grupos de candidatos con formas irregulares y para detectar "ruido" (perfiles falsos o incompletos) que no encajan en ningún clúster.

Análisis de componentes principales (PCA)

Reducción de dimensionalidad: Un CV puede tener cientos de variables (cada palabra o habilidad es una dimensión). PCA reduce esta complejidad transformando las variables correlacionadas en un número menor de "componentes principales" sin perder información relevante, lo que acelera el procesamiento del agente y evita el "overfitting".

4. Procesamiento de lenguaje natural (NLP) y preprocesamiento

Fuentes de datos y estructura

El sistema se alimenta de datos no estructurados provenientes de currículums (PDF, DOCX) y descripciones de vacantes. El reto es convertir este texto libre en datos estructurados (JSON) procesables por los algoritmos.

Técnicas de NLP para extracción de características

Antes de analizar, el texto debe limpiarse:

- Tokenización: Dividir el texto en unidades (palabras/frases).
- Stop Word Removal: Eliminar palabras sin valor semántico (ej. "el", "de", "para").
- Stemming/Lemmatization: Reducir palabras a su raíz (ej. "programando", "programado" -> "program") para agrupar conceptos similares.

Modelos de representación de texto (embeddings)

Para que el algoritmo matemático entienda el texto, este debe convertirse en números (vectores).

- Word Embeddings (Word2Vec, GloVe): Representan palabras en un espacio vectorial donde palabras con significados similares están cerca geométricamente.
- Transformers (BERT/GPT): A diferencia de Word2Vec, los modelos basados en Transformers (como la API de Google CLI usada en el proyecto) entienden el contexto completo de una frase, permitiendo distinguir entre "Java" (lenguaje) y "Java" (isla) o entender matices de liderazgo en una descripción de experiencia.

5. Evaluación de desempeño y mantenimiento

Métricas de evaluación para clustering

Dado que no hay etiquetas "correctas" en el aprendizaje no supervisado, se usan métricas de calidad interna:

- Coeficiente de silueta: Mide qué tan similar es un candidato a su propio clúster en comparación con otros. Un valor cercano a 1 indica una segmentación perfecta.
- Índice de Davies-Bouldin: Evalúa la separación entre clústeres; un valor más bajo indica una mejor diferenciación entre los grupos de candidatos.

Validación y optimización

El ajuste de hiperparámetros (ej. definir el número óptimo K de clústeres) es vital para asegurar que la segmentación tenga sentido para el negocio y no sea puramente matemática.

Estrategias de integración y monitoreo

Una vez en producción, el agente debe ser monitoreado para detectar la "degradación del modelo" (Data Drift), donde los cambios en el mercado laboral (nuevas habilidades) hacen que el modelo pierda precisión. La arquitectura basada en APIs (Google CLI) facilita la actualización continua del núcleo de IA sin detener el orquestador.

Arquitectura de Sistemas Multi-Agente y Orquestación en Software

Para comprender el desarrollo de soluciones modernas de HR Tech, es necesario ir más allá de los algoritmos aislados y entender la arquitectura que los gobierna. En el contexto de este proyecto, se aplican conceptos de Sistemas Multi-Agente (SMA) y orquestación de servicios.

Sistemas Multi-Agente (SMA) y Automatización Agéntica

Un sistema multiagente se define como una red de entidades de software inteligentes que interactúan para resolver problemas que son difíciles o imposibles para un agente monolítico individual. En la automatización de procesos con agentes (Agentic Process Automation), cada módulo o "agente" tiene una responsabilidad específica (percepción,

segmentación, comunicación) y opera de manera autónoma o semi-autónoma (Automation Anywhere, 2025).

En el ámbito de reclutamiento, esta arquitectura permite desacoplar funciones: un agente puede encargarse exclusivamente de la calendarización mientras otro analiza riesgos en el perfil del candidato, permitiendo una escalabilidad modular que los sistemas tradicionales (monolíticos) no ofrecen (Google Cloud, 2025).

1.3.2. El Rol del Orquestador en la Integración de Servicios

A diferencia de una simple coreografía donde los servicios interactúan caóticamente entre sí, la "orquestación" implica un control centralizado que coordina la ejecución de flujos de trabajo. El orquestador actúa como un director que gestiona las dependencias, asegura que los datos fluyan en el orden correcto (por ejemplo, no analizar riesgos antes de segmentar al candidato) y maneja los errores de ejecución (Xygeni, 2023).

En el desarrollo de software para RR.HH., el orquestador valida que las reglas de negocio se cumplan antes de pasar a la siguiente etapa, garantizando la integridad del proceso de selección automatizado (IBM, 2025).

Heurísticas en la Evaluación de Candidatos

Aunque la Inteligencia Artificial generativa es tendencia, gran parte de la selección automatizada efectiva se basa en "heurísticas" o reglas lógicas de decisión. Una heurística es un atajo mental o computacional que permite tomar decisiones rápidas basadas en criterios predefinidos (como umbrales de puntuación o palabras clave críticas) (Fortinet, 2025).

En sistemas expertos de reclutamiento, las heurísticas permiten traducir la intuición del reclutador (ej. "si tiene menos de 2 años de experiencia, descartar") en algoritmos deterministas. Esto reduce la "sobrecarga cognitiva" del evaluador humano y asegura que todos los candidatos sean medidos bajo la misma vara técnica antes de pasar a etapas subjetivas (Genomawork, 2022).

CAPÍTULO 2.

DESARROLLO

1. Metodología de Desarrollo y Análisis Inicial

Para la ejecución de este proyecto en Plurione S.A. de C.V., no seguimos un desarrollo lineal tradicional. Debido a la naturaleza experimental de integrar inteligencia artificial con un sistema legado en PHP, opté por una metodología incremental. Mi prioridad inicial fue entender cómo "romper" el monolito existente para inyectarle lógica moderna sin tirar todo el sistema.

El análisis inicial reveló que el sistema de reclutamiento operaba con scripts dispersos. Mi primer gran reto técnico fue diseñar una arquitectura que permitiera la convivencia de múltiples agentes. No podía simplemente poner "if" en el código; necesitaba un Orquestador Central.

Para lograr esto, dividí el desarrollo en dos grandes módulos:

- Sistema Web Transaccional (Core): Programación de todas las vistas y lógica de negocio para usuarios y administradores.
- Módulo de Inteligencia Artificial: Integración de agentes inteligentes sobre el sistema base que construí.

2. Diseño de la arquitectura del backend

Basándome en los patrones de diseño de software, propuse la creación de una clase central llamada Agente Orquestador (`agente_orquestador.php`). Esta pieza de software es el cerebro del proyecto.

La lógica que implementé funciona así:

- El sistema recibe una petición del Frontend (ej. "Evaluar candidato").
- El Orquestador intercepta la petición.
- Decide qué "especialista" (Agente) debe actuar: ¿Segmentación? ¿Riesgos? ¿Validación?
- Ejecuta el agente y guarda un registro estricto de lo sucedido.

Como se puede observar en el fragmento de código de la Imagen 1, la estructura del orquestador utiliza un switch para determinar qué acción ejecutar (segmentación, riesgos, validación, etc.), asegurando que el sistema sea modular y escalable.

```
public function __construct($mysqli) {
    $this->mysqli = $mysqli;
}

/**
 * Ejecuta un agente específico
 * @param string $nombre_agente Nombre del agente ('segmentacion', 'deteccion_riesgos', etc.)
 * @param array $datos Datos necesarios para el agente
 * @return array Resultado de la ejecución
 */
public function ejecutarAgente($nombre_agente, $datos = []) {
    $id_log = $this->registrarInicio($nombre_agente, $datos);
    $resultado = ['success' => false, 'mensaje' => ''];

    try {
        switch($nombre_agente) {
            case 'segmentacion':
                $resultado = $this->ejecutarSegmentacion($datos);
                break;

            case 'deteccion_riesgos':
                $resultado = $this->ejecutarDeteccionRiesgos($datos);
                break;

            case 'calendarizacion':
                $resultado = $this->ejecutarCalendarizacion($datos);
                break;

            case 'feedback_rechazo':
                $resultado = $this->ejecutarFeedbackRechazo($datos);
                break;

            case 'validacion_proceso':
                require_once 'agente_validacion_proceso.php';
                $agente = new AgenteValidacionProceso($this->mysqli);
                return $agente->validar($datos['id_aplicacion']);

            case 'seguimiento_ingreso':
                require_once 'agente_seguimiento_ingreso.php';
                $agente = new AgenteSeguimientoIngreso($this->mysqli);
                $accion = $datos['accion'] ?? 'iniciar';
                if ($accion === 'iniciar') {
```

Imagen 1. Lógica de control centralizada en la clase AgenteOrquestador de Autoría propia (2025).

Esta arquitectura fue fundamental para pasar de un sistema de "archivos sueltos" a una plataforma organizada donde cada agente tiene una responsabilidad única.

3. Diseño e Implementación de la base de datos

Uno de los puntos más fuertes de mi proyecto fue el diseño de la base de datos recursosh. Durante el desarrollo, me di cuenta de que una base de datos relacional estándar (MySQL)

se quedaría corta para almacenar el "pensamiento" de una IA. Por eso, diseñé un esquema híbrido que aprovecha campos JSON para dar flexibilidad a tablas rígidas.

Las principales estructuras que desarrollé e implementé en SQL son:

- Tabla log_agentes (La memoria del sistema): Esta tabla es crítica para la auditoría. Diseñé columnas para guardar datos_entrada y datos_salida en formato JSON. Esto me permite, como desarrollador, ver exactamente qué "leyó" el agente y qué respondió, facilitando enormemente la depuración de errores en la lógica de la IA.

id_log	id_aplicacion	agente_nombre	estado	datos_entrada	datos_salida	fecha_inicio	fecha_fin	error_mensaje
1	11	segmentacion	procesando	{"id_aplicacion":11}	NULL	2025-12-03 11:57:10	NULL	NULL
2	12	segmentacion	procesando	{"id_aplicacion":12}	NULL	2025-12-03 12:01:37	NULL	NULL
3	1	calendarizacion	completado	{"accion":"crear","id_aplicacion":"1","fecha_base"...	{"success":true,"id_entrevista":1,"opciones":[{"fe...	2025-12-03 12:30:14	2025-12-03 12:30:14	NULL
4	12	feedback_rechazo	procesando	{"id_aplicacion":"12","razones":["Score bajo"],"No ...	NULL	2025-12-03 12:30:38	NULL	NULL
5	12	feedback_rechazo	completado	{"id_aplicacion":"12","razones":["Score bajo"],"No ...	{"success":true,"mensaje":"Feedback generado y env...	2025-12-03 12:33:18	2025-12-03 12:33:19	NULL
6	1	calendarizacion	completado	{"accion":"crear","id_aplicacion":"1","fecha_base"...	{"success":true,"id_entrevista":1,"opciones":[{"fe...	2025-12-03 12:36:21	2025-12-03 12:36:21	NULL
7	12	validacion_proceso	procesando	{"id_aplicacion":"12"}	NULL	2025-12-03 17:05:31	NULL	NULL
8	11	validacion_proceso	procesando	{"id_aplicacion":"11"}	NULL	2025-12-03 17:05:33	NULL	NULL
9	12	validacion_proceso	procesando	{"id_aplicacion":"12"}	NULL	2025-12-03 17:05:38	NULL	NULL
10	2	validacion_proceso	procesando	{"id_aplicacion":"2"}	NULL	2025-12-03 17:08:43	NULL	NULL

Imagen 2. Estructura de la tabla de auditoría log_agentes de Autoría propia(2025).

- Tabla evaluaciones (Soporte al agente de segmentación): Para mi agente principal, creé esta tabla con campos calculados. No solo guarda un número; implementé un campo ENUM llamado clasificacion_fit con valores 'Alto Fit', 'Medio Fit', 'Bajo Fit'. Esto permite que el backend clasifique automáticamente al candidato y lo muestre así en la vista del reclutador, eliminando la necesidad de interpretación manual.

#	Nombre	Tipo	Cotejamiento	Atributos	Nulo	Predeterminado	Comentarios	Extra	Acción
☐	1 id_evaluacion	int(11)			No	Ninguna		AUTO_INCREMENT	Cambiar Eliminar Más
☐	2 id_aplicacion	int(11)			No	Ninguna			Cambiar Eliminar Más
☐	3 score_tecnico	int(3)			Sí	0			Cambiar Eliminar Más
☐	4 score_blando	int(3)			Sí	0			Cambiar Eliminar Más
☐	5 score_global	int(3)			Sí	NULL		VIRTUAL GENERATED	Cambiar Eliminar Más
☐	6 clasificacion_fit	enum('Alto Fit', 'Medio Fit', 'Bajo Fit')	utf8mb4_general_ci		Sí	NULL			Cambiar Eliminar Más
☐	7 comentarios_tecnicos	text	utf8mb4_general_ci		Sí	NULL			Cambiar Eliminar Más
☐	8 fecha_evaluacion	timestamp			No	current_timestamp()			Cambiar Eliminar Más

Imagen 3. Registros de evaluaciones con clasificación automática de "Fit" del candidato de auditoria propia (2025).

- Tablas de lógica difusa (riesgos_detectados y feedback_rechazo): Para manejar datos cualitativos, creé tablas especializadas. En riesgos_detectados, el sistema almacena banderas rojas como gap_temporal o inconsistencia. En

feedback_rechazo, utilicé nuevamente un campo JSON razones_rechazo para guardar listas dinámicas de motivos por los que un candidato no fue aceptado, permitiendo generar correos personalizados automáticamente.

- Tablas de Estado (onboarding y comentarios_validacion): Para los procesos administrativos, diseñé tablas que funcionan como máquinas de estados. Por ejemplo, onboarding controla el progreso del nuevo empleado mediante columnas como doc_contratacion y config_equipos, que actualizan visualmente las barras de progreso en el dashboard.

4. Desarrollo de componentes y agentes

Una vez lista la base de datos, procedí a la codificación de los agentes utilizando Gemini CLI para definir sus personalidades y reglas de negocio, y PHP para integrarlos al sistema.

Desarrollo del Agente de Segmentación (ASAC)

Este fue mi proyecto núcleo. Programé la lógica en procesar_fit.php para que se disparara automáticamente tras una postulación. El algoritmo toma el CV y las respuestas del candidato, las procesa, y escribe directamente en la tabla evaluaciones los puntajes técnicos y blandos. Fue muy satisfactorio ver cómo el sistema pasaba de simplemente "guardar un PDF" a "entender el perfil" del candidato en segundos.

Integración de agentes satélite

Aunque mi foco era la segmentación, para que el sistema fuera funcional tuve que integrar otros agentes:

- Agente de Riesgos y feedback: Lo configuré para ejecutarse en paralelo a la segmentación, buscando anomalías en los datos. Para manejar datos cualitativos, integré agentes que escriben en tablas especializadas. Por ejemplo, el agente de feedback genera razones de rechazo dinámicas que se almacenan en la tabla feedback_rechazo, permitiendo personalizar las respuestas automáticas a los candidatos no seleccionados.
- Agente de Validación (Cliente Interno): Creé la lógica para conectar la vista admin_validacion.php. Ahora, cuando un gerente aprueba o rechaza, el agente

registra la acción en la base de datos y cambia el estado del flujo. El código mostrado en la imagen 2.4 evidencia cómo se instancia la clase `AgenteValidacionProceso` para manejar estados como "aprobado" o "rechazado".

```
// Validar CV
if (empty($datos['cv_path']) || !file_exists($datos['cv_path'])) {
    $validaciones[] = [
        'tipo' => 'entregable',
        'estado' => 'rechazado',
        'descripcion' => 'CV no encontrado',
        'fecha_limite' => date('Y-m-d', strtotime('+3 days'))
    ];
}

// Validar evaluación
if (!$datos['score_global'] || $datos['score_global'] < 30) {
    $validaciones[] = [
        'tipo' => 'proceso',
        'estado' => 'rechazado',
        'descripcion' => 'Evaluación incompleta o score muy bajo',
        'fecha_limite' => date('Y-m-d', strtotime('+1 day'))
    ];
}

// Si todo está bien
if (empty($validaciones)) {
    $validaciones[] = [
        'tipo' => 'proceso',
        'estado' => 'aprobado',
        'descripcion' => 'Proceso validado correctamente',
        'fecha_limite' => null
    ];
}
```

Imagen 4. Implementación de la clase para el Agente de Validación de Procesos de auditoría propia (2025).

5. Integración Frontend-Backend (Vistas Administrativas)

La última fase fuerte del desarrollo fue "conectar los cables". El equipo de Frontend entregó las vistas estáticas (`admin_evaluacion.php`, `admin_onboarding.php`), y mi tarea fue darles vida con PHP.

- Panel de Evaluación: Conecté los botones de acción para que no fueran estáticos. Al dar clic en "Contratar", programé el backend para que active el Agente de Onboarding, creando automáticamente el registro en la tabla `onboarding` y asignando las tareas pendientes iniciales.
- Panel de Validación: Implementé la lógica para que los comentarios de los gerentes se guarden en `comentarios_validacion` y sean visibles para RRHH en tiempo real.
- Panel de Onboarding: Conecté la vista administrativa `admin_onboarding.php` para que reflejara en tiempo real el progreso de los nuevos ingresos. Como se visualiza en la interfaz del sistema en la Figura 2.5, el panel muestra el estado de tareas

críticas (documentación, equipos, inducción) que son alimentadas directamente desde la tabla onboarding.

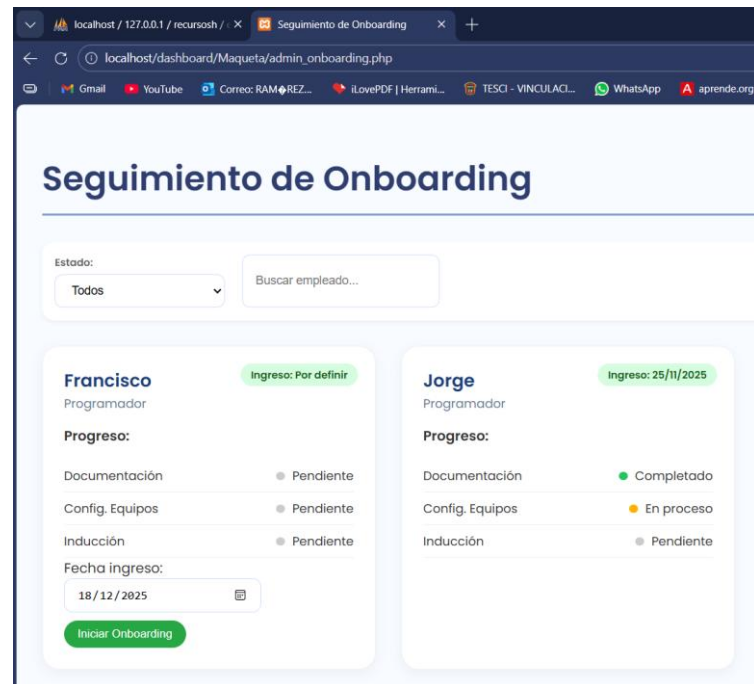


Imagen 5 Vista administrativa del módulo de Onboarding con estados actualizados en tiempo real de Autoría propia (2025).

6. Implementación del módulo de autenticación y seguridad

Una vez definida la arquitectura base, procedí a integrar la capa de seguridad para gestionar el acceso de los usuarios, tal como se especifica en la sección de autenticación de la documentación maestra. Para ello, trabajé sobre el controlador `login_proceso.php`, que actúa como la puerta de entrada al sistema.

En este archivo implementé la lógica de validación de credenciales. Utilicé la función nativa `password_verify()` de PHP para comparar de manera segura las contraseñas ingresadas contra los registros encriptados en la base de datos. Además, configuré el manejo de sesiones mediante `session_start()`, lo que permite al sistema mantener la persistencia del usuario y redirigirlo a su vista correspondiente (`inicio.php` para candidatos o el panel administrativo para RRHH) según su rol, restringiendo así el acceso a áreas no autorizadas.

```
// 3. Verificar si el email existe
if ($stmt->num_rows == 1) {
    // Asegúrate de que el primer parámetro en bind_result coincide con la columna 'id' de la BD
    $stmt->bind_result($id, $email, $hashed_password);

    if ($stmt->fetch() && !is_null($hashed_password) && is_string($hashed_password)) {
        // 4. Verificar la contraseña
        if (password_verify($password_login, $hashed_password)) {
            // ¡Inicio de sesión exitoso!
            // 5. Asignación de la Sesión Estándar
            $_SESSION["loggedin"] = true;
            $_SESSION["id"] = $id;
            $_SESSION["email"] = $email;

            // Redirigir al dashboard
            header("location: inicio.php");
            exit;
        } else {
            $login_err = "La contraseña que has ingresado no es válida.";
        }
    }
} else {
    $login_err = "No existe una cuenta con ese correo electrónico.";
}
} else {
    $login_err = "Oops! Algo salió mal. Por favor, inténtalo de nuevo más tarde.";
}

$stmt->close();
```

Imagen 6 Controlador de inicio de sesión y validación de usuarios de Autoría propia (2025).

7. Desarrollo de la interfaz de postulación y seguimiento

Como parte de la integración del sistema, desarrollé las vistas que permiten al candidato interactuar con el backend. Me enfoqué en el archivo vacantes.php, el cual programé para consultar y mostrar dinámicamente las oportunidades laborales almacenadas en la base de datos.

El punto crítico de esta integración se encuentra en el script aplicar.php. Programé este controlador para que, al recibir una solicitud de postulación, no solo almacene la información del candidato, sino que funcione como el detonador del proceso de inteligencia artificial. Es en este punto donde el sistema invoca al Agente de Segmentación (descrito en la sección 2.4) para que analice el perfil en tiempo real y genere los puntajes de compatibilidad iniciales.

Finalmente, para brindar transparencia al usuario, implementé la vista aplicaciones.php. Esta interfaz consulta el estado actual de cada postulación y lo presenta visualmente, permitiendo al candidato saber si su solicitud se encuentra en etapa de "Evaluación", "Entrevista" o "Finalizado", cerrando así el ciclo de información entre el backend y el usuario.



Imagen 7 Interfaz de usuario para la visualización de vacantes y seguimiento de estado de Autoría Propia (2025).

8. Integración avanzada de agentes y correcciones del sistema

Con el sistema funcionando con algunos agentes conectados, me entregaron una nueva parte del sistema que necesitaba integrar, mi trabajo se dividió en dos fases claramente diferenciadas.

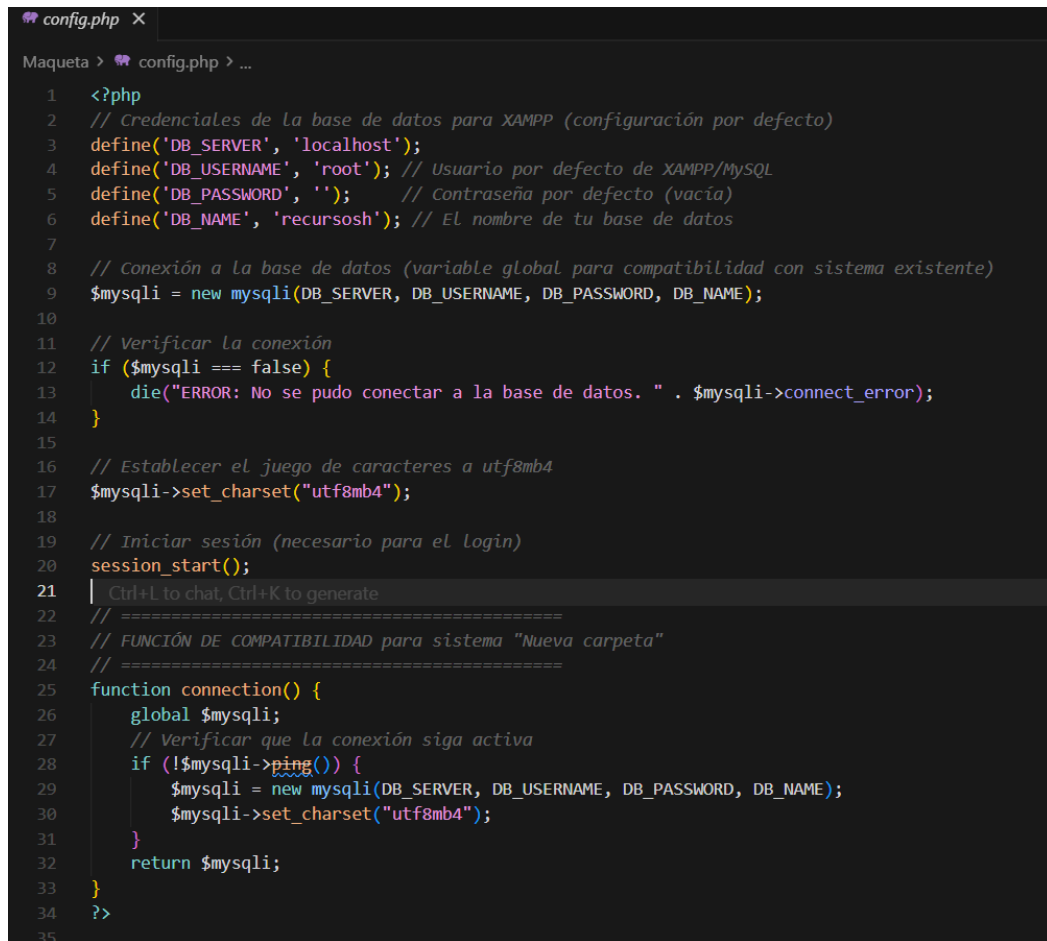
En la primera fase, me dediqué a adaptar completamente la nueva parte del sistema entregada para que funcionara sobre mi base de datos recursosh y se integrara con mi sistema existente. Esto implicó modificar el archivo config.php ya que es el punto central de configuración para unificar las conexiones, desarrollar funciones helper para mapear las diferencias entre esquemas, y asegurar que todas las consultas utilizaran los nombres correctos de tablas y columnas según mi estructura de datos. Fue un trabajo meticuloso que requirió analizar cada consulta SQL y adaptarla sin romper la funcionalidad existente de mi sistema.

Una vez que la nueva parte del sistema funcionaba correctamente sobre mi base de datos e integrada con mi sistema existente, comencé la segunda fase: conectar los agentes que faltaban. Fue en este punto donde me enfrenté al reto más complejo: hacer que todos estos componentes trabajaran de manera cohesiva con el sistema ya adaptado. No bastaba con que cada agente funcionara individualmente; necesitaba que el flujo completo fuera robusto y manejara correctamente las relaciones entre tablas, tanto las que ya existían como las nuevas que se agregaron.

Adaptación de la nueva parte del sistema

La primera etapa de mi trabajo fue adaptar la nueva parte del sistema entregada para que funcionara con mi base de datos y se integrara con mi sistema existente. Esto requirió analizar profundamente las diferencias entre los esquemas esperados por el código entregado y mi estructura real de base de datos recursosh. Fue un proceso iterativo donde tuve que identificar cada incompatibilidad y encontrar una solución que no rompiera el código existente de mi sistema, que ya estaba funcionando en producción.

El punto central de esta adaptación fue la modificación del archivo config.php, que se convirtió en el corazón de toda la integración. En este archivo, implementé una función connection() que permite al nuevo sistema obtener la conexión de forma compatible, mientras mantengo la variable global \$mysqli para el código existente. Esta función de compatibilidad fue crucial porque permite que tanto mi código existente (que usa \$mysqli directamente) como el nuevo código (que usa connection()) compartan la misma conexión, garantizando consistencia en las transacciones y evitando problemas de sesiones duplicadas:



```

1  <?php
2  // Credenciales de la base de datos para XAMPP (configuración por defecto)
3  define('DB_SERVER', 'localhost');
4  define('DB_USERNAME', 'root'); // Usuario por defecto de XAMPP/MySQL
5  define('DB_PASSWORD', ''); // Contraseña por defecto (vacía)
6  define('DB_NAME', 'recursosh'); // El nombre de tu base de datos
7
8  // Conexión a la base de datos (variable global para compatibilidad con sistema existente)
9  $mysqli = new mysqli(DB_SERVER, DB_USERNAME, DB_PASSWORD, DB_NAME);
10
11 // Verificar la conexión
12 if ($mysqli === false) {
13     die("ERROR: No se pudo conectar a la base de datos. " . $mysqli->connect_error);
14 }
15
16 // Establecer el juego de caracteres a utf8mb4
17 $mysqli->set_charset("utf8mb4");
18
19 // Iniciar sesión (necesario para el login)
20 session_start();
21 | Ctrl+L to chat, Ctrl+K to generate
22 // =====
23 // FUNCIÓN DE COMPATIBILIDAD para sistema "Nueva carpeta"
24 // =====
25 function connection() {
26     global $mysqli;
27     // Verificar que la conexión siga activa
28     if (!$mysqli->ping()) {
29         $mysqli = new mysqli(DB_SERVER, DB_USERNAME, DB_PASSWORD, DB_NAME);
30         $mysqli->set_charset("utf8mb4");
31     }
32     return $mysqli;
33 }
34 ?>
35

```

Imagen 8. Archivo config.php modificado de Autoria propia (2025)

Además, implementé un sistema de helpers en helpers_db.php que centralizaba las consultas adaptadas, utilizando alias SQL para crear nombres de columnas compatibles. Por ejemplo, la nueva parte del sistema esperaba columnas como idClientes y NombreCompleto, pero mi base de datos tenía id y nombre. La solución fue usar alias SQL que hacen la traducción transparente. Para casos más complejos, como las vacantes, utilicé la función COALESCE para manejar diferencias de nombres de columnas de manera flexible. Un caso particularmente interesante fue la transformación del campo salario, que podía venir como string con formato “\$40,000 - \$60,000” pero el sistema necesitaba un número para cálculos. Implementé una transformación que limpia los caracteres no numéricos y convierte a entero automáticamente.

```
function obtenerVacantesCompatibles($conn) {  
    $sql = "SELECT  
        id_vacante AS idVacante,  
        titulo,  
        COALESCE(departamento, empresa) AS departamento,  
        COALESCE(tipo, tipo_trabajo) AS tipo,  
        ubicacion,  
        descripcion,  
        requisitos,  
        CAST(REPLACE(REPLACE(REPLACE(salario, '$', ''), ',', ''), ' ', '' ) AS UNSIGNED) AS salario,  
        COALESCE(fechaApertura, fecha_publicacion) AS fechaApertura,  
        COALESCE(fechaCierre, fecha_publicacion) AS fechaCierre,  
        COALESCE(responsable, 'RRHH') AS responsable,  
        estado  
    FROM vacantes  
    ORDER BY id_vacante DESC";  
    return $conn->query($sql);  
}
```

Imagen 9 fragmento de código helpers_db.php de Autoría propia (2025)

Esta adaptación inicial fue fundamental porque sin ella, ni siquiera podía empezar a pensar en conectar los agentes faltantes. Una vez que la nueva parte del sistema funcionaba correctamente sobre mi base de datos e integrada con mi sistema existente, con todas sus vistas, formularios y funcionalidades básicas operativas, pude enfocarme en la segunda fase: conectar los agentes que faltaban. Fue muy satisfactorio ver cómo el sistema pasaba de tener agentes funcionando en una parte a tener una plataforma completamente unificada donde todos los agentes trabajaban de manera cohesiva. El resultado de esta integración se puede observar claramente en el sistema funcionando, donde todas las funcionalidades trabajan de manera integrada sin problemas de compatibilidad.

Resolución de restricciones de Foreign keys

Uno de los primeros problemas que encontré al conectar los agentes faltantes a la nueva parte del sistema fue al intentar programar entrevistas desde el panel administrativo. El sistema lanzaba errores de foreign key constraint porque intentaba insertar registros en la tabla entrevistas sin que existiera previamente un registro en aplicaciones. Este error me hizo darme cuenta de que necesitaba pensar más cuidadosamente en el orden de las operaciones, especialmente al integrar la nueva funcionalidad con los agentes existentes.

Para resolverlo, implementé una lógica de verificación y creación automática que se ejecuta antes de cualquier inserción crítica. El sistema ahora verifica automáticamente si existe una aplicación para el candidato y la vacante, y si no existe, la crea de forma

transparente antes de programar la entrevista. Esto garantiza la integridad referencial sin requerir que el usuario realice pasos adicionales manualmente. Fue un cambio arquitectónico pequeño pero crítico que mejoró significativamente la experiencia de uso, como se puede observar en la interfaz del sistema cuando se programa una entrevista desde el panel administrativo.



Imagen 10. Panel de programación de entrevistas funcionando correctamente tras la resolución de restricciones de foreign keys de Autoría propia (2025).

Manejo de restricciones únicas y duplicados

Otro desafío técnico importante que enfrenté fue la restricción única en la columna `id_aplicacion` de la tabla `entrevistas`. El diseño de la base de datos permitía solo una entrevista por aplicación, lo cual tenía sentido desde el punto de vista del negocio, pero complicaba la lógica de actualización. Me di cuenta de que necesitaba una estrategia diferente a simplemente intentar insertar y manejar el error.

En lugar de modificar la estructura de la base de datos (lo cual habría roto otras partes del sistema), implementé una estrategia de “verificar antes de insertar, actualizar si existe”. El sistema ahora detecta automáticamente si ya existe una entrevista para una aplicación y la

actualiza en lugar de intentar crear un duplicado. Esta solución, aunque aparentemente simple, requirió cambios en múltiples puntos del código tanto del agente de calendarización como del panel administrativo. Fue un buen recordatorio de la importancia de pensar en los casos límite desde el diseño inicial, no solo después de que

aparecen los errores. El resultado es que el sistema maneja elegantemente las actualizaciones de entrevistas sin errores, como se puede verificar en el panel de administración.

id_entrevista	id_cliente	id_vacante	id_reclutador	id_aplicacion	fecha_propuesta_1	hora_propuesta_1	fecha_propuesta_2	hora_propuesta_2	fecha_propuesta_3	hora_propuesta_3	fecha_final	hora_final	fecha	hora_inicio	hora_fin	estado	notas	fecha_creacion	status_confirmacion
	PK candidato	PK vacante id_vacante	PK reclutador id_reclutador										fecha final confirmada	hora inicio	hora fin				
1	NULL	NULL	NULL	1	2025-12-06	10:00:00	2025-12-07	14:00:00	2025-12-08	16:00:00	2025-11-04	11:00:00	NULL	NULL	NULL	Programada	NULL	2025-12-17 13:03:43	Pendiente de confirmación
2	NULL	NULL	NULL	2	2025-11-20	10:00:00	2025-11-21	14:00:00	2025-11-22	16:00:00	2025-11-17	13:00:00	NULL	NULL	NULL	Programada	NULL	2025-12-17 13:03:43	Confirmada
3	NULL	NULL	NULL	3	2025-12-05	09:00:00	2025-12-06	12:00:00	2025-12-07	17:00:00	NULL	NULL	NULL	NULL	NULL	Programada	NULL	2025-12-17 13:03:43	Pendiente de confirmación
4	NULL	NULL	NULL	6	2026-01-01	12:41:03	2026-01-08	13:41:03	2026-01-07	12:41:03	2026-01-16	17:41:03	NULL	NULL	NULL	Programada	NULL	2025-12-17 13:03:43	Pendiente de confirmación
10	8	1	1	11	2025-12-21	10:00:00	2025-12-22	14:00:00	2025-12-23	16:00:00	NULL	NULL	0000-00-00	18:19:00	18:23:00	Programada	Entrevista programada desde el panel.	2025-12-17 16:19:23	Pendiente de confirmación
16	8	4	1	13	0000-00-00	00:00:00	NULL	NULL	NULL	NULL	NULL	NULL	0000-00-00	09:00:00	10:00:00	Programado por IA	Entrevista programada desde el panel.	2025-12-17 19:44:08	Pendiente de confirmación
19	8	5	1	19	2025-12-21	10:00:00	2025-12-22	14:00:00	2025-12-23	16:00:00	NULL	NULL	0000-00-00	09:00:00	10:00:00	Programado por IA	NULL	2025-12-17 21:00:39	Pendiente de confirmación
20	1	5	1	20	2025-12-21	10:00:00	2025-12-22	14:00:00	2025-12-23	16:00:00	NULL	NULL	2025-12-18	00:07:00	13:11:00	Programada	Entrevista programada desde el panel.	2025-12-17 21:02:51	Pendiente de confirmación

Imagen 11. Sistema de entrevistas funcionando sin errores de duplicados tras implementar la verificación previa de Autoría propia (2025).

Implementación del agente de feedback automatizado

Uno de los agentes más complejos que implementé fue el sistema de feedback automatizado para candidatos rechazados. Este agente tiene dos modos de operación: procesamiento individual (cuando un reclutador rechaza manualmente a un candidato desde el panel de evaluación) y procesamiento en lote (mediante un cron job que procesa todos los rechazados pendientes).

Para el modo individual, diseñé el flujo de manera que la actualización del status siempre ocurra primero, garantizando que el candidato quede marcado como rechazado incluso si el agente de feedback falla. Esta separación de responsabilidades me permitió garantizar que las operaciones críticas (como cambiar el estado) nunca dependieran del éxito de operaciones secundarias (como generar un mensaje automático). Aprendí mucho sobre diseño de sistemas resilientes durante este proceso.

El agente genera automáticamente mensajes personalizados con las razones del rechazo y sugerencias de mejora, que se almacenan en la tabla `feedback_rechazo` y se registran en el canal de comunicación. Como se puede observar en la interfaz del panel de evaluación, cuando un reclutador rechaza a un candidato, el sistema procesa automáticamente el feedback sin interrumpir el flujo de trabajo.

id_feedback	id_aplicacion	mensaje_generado	razones_rechazo	sugerencias_mejora	fecha_envio	estado_envio
1	12	Hola Santiago,	["Score bajo", "No cumple requisitos"]	Continúa desarrollando tu experiencia y habilidade...	2025-12-03 12:33:19	enviado
		Gracias por tu interés en la posic...				
2	14	Hola Santiago,	["Score bajo", "No cumple requisitos"]	Continúa desarrollando tu experiencia y habilidade...	2025-12-17 20:06:58	enviado
		Gracias por tu interés en la posic...				
3	1	Hola Francisco,	["No cumple con los requisitos m\u00e9dicos del pu..."]	Continúa desarrollando tu experiencia y habilidade...	2025-12-17 20:12:50	enviado
		Gracias por tu interés en la posic...				
4	9	Hola Jorge,	["No cumple con los requisitos m\u00e9dicos del pu..."]	Continúa desarrollando tu experiencia y habilidade...	2025-12-17 20:12:50	enviado
		Gracias por tu interés en la posición...				
5	16	Hola Marco ,	["Score bajo", "No cumple requisitos"]	Continúa desarrollando tu experiencia y habilidade...	2025-12-17 20:15:06	enviado
		Gracias por tu interés en la posició...				

Imagen 12 base de datos mostrando el proceso automático de feedback al rechazar un candidato de Autoría propia (2025).

Mejora del agente de detección de riesgos

Durante las pruebas del sistema, me di cuenta de que el agente de detección de riesgos no estaba detectando casos evidentes como candidatos que ingresaban “no se” o “no me importa” en el campo de habilidades técnicas. Esto me hizo reflexionar sobre cómo la IA necesita reglas explícitas para ciertos casos que, para un humano, son obvios. Para mejorar esto, implementé una lista de palabras sospechosas que el sistema busca activamente.

Además, mejoré la detección de CVs faltantes implementando una verificación de existencia de archivos que maneja tanto rutas absolutas como relativas. El sistema ahora detecta múltiples tipos de riesgos: scores extremadamente bajos, habilidades técnicas sospechosas o vacías, información de contacto incompleta, y CVs no subidos. Estas mejoras convirtieron al agente de detección de riesgos en una herramienta mucho más efectiva para identificar aplicaciones problemáticas antes de que consumieran tiempo del equipo de reclutamiento. Los riesgos detectados se muestran claramente en el panel de

evaluación, con indicadores de severidad (alta, media, baja) que ayudan al reclutador a priorizar su revisión.

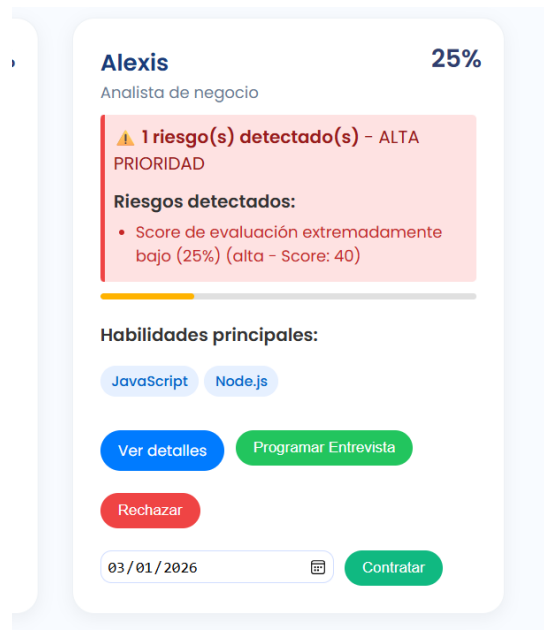


Imagen 13 Panel de evaluación mostrando riesgos detectados automáticamente por el agente con indicadores de severidad de Autoría propia (2025).

Corrección del agente de segmentación

Uno de los errores más sutiles que encontré fue al intentar insertar el `score_global` directamente en la tabla evaluaciones. Pasé varias horas depurando este problema antes de descubrir que esta columna estaba definida como `GENERATED` (calculada automáticamente por MySQL). Fue frustrante al principio, pero me enseñó la importancia de conocer profundamente el esquema de la base de datos. La solución fue simplemente omitir este campo del `INSERT`, confiando en que la base de datos lo calcule automáticamente.

Este error me enseñó la importancia de conocer profundamente el esquema de la base de datos, no solo los nombres de las tablas, sino también sus propiedades y restricciones. A veces, la solución más simple es la correcta, pero primero hay que entender completamente el problema. Una vez corregido, el agente de segmentación funciona correctamente, calculando los scores técnico y blando, asignando el segmento (A, B, C) y mostrando los resultados en el panel de evaluación del sistema.

id_evaluacion	id_aplicacion	score_tecnico	score_blando	score_global	clasificacion_fit	segmento	comentarios_tecnicos	fecha_evaluacion
1	1	1	95	80	88 NULL	C	Excelente nivel de JS, le falta un poco de trabajo...	2025-11-24 11:11:11
2	17	0	50	25	Bajo Fit	C	Análisis Automático: Se detectaron coincidencias e...	2025-12-17 20:29:53
3	18	0	50	25	Bajo Fit	C	Análisis Automático: Se detectaron coincidencias e...	2025-12-17 20:51:31
4	19	60	50	55	Medio Fit	C	Análisis Automático: Se detectaron coincidencias e...	2025-12-17 21:00:14
5	20	100	50	75	Alto Fit	B	Análisis Automático: Se detectaron coincidencias e...	2025-12-17 21:02:39
6	21	0	50	25	Bajo Fit	C	Análisis Automático: Se detectaron coincidencias e...	2025-12-19 14:40:43
7	22	0	50	25	Bajo Fit	C	Análisis Automático: Se detectaron coincidencias e...	2025-12-27 15:32:38

Imagen 14. Panel de evaluación mostrando los scores calculados automáticamente por el agente de segmentación de Autoría propia (2025).

Integración del Agente de Calendarización con interfaz de usuario

El agente de calendarización fue particularmente complejo porque no solo necesitaba funcionar correctamente en el backend, sino que también requería interactuar dinámicamente con la interfaz de usuario cuando un candidato rechazaba un horario propuesto.

Implementé un sistema de historial de propuestas y un índice actual que permite al candidato navegar entre diferentes opciones de horarios. Cuando un candidato rechaza una propuesta, el sistema genera automáticamente una nueva opción y actualiza la interfaz de manera instantánea. Esta integración entre PHP y JavaScript me permitió crear una experiencia de usuario fluida donde el sistema responde inmediatamente a las acciones del candidato sin requerir recargas completas de página.

El agente utiliza una estrategia de tres fases para encontrar horarios: primero busca coincidencias exactas entre la disponibilidad del candidato y el reclutador, luego busca opciones donde el candidato tiene disponibilidad y el reclutador puede ajustarse, y finalmente genera horarios extendidos basados en la disponibilidad del reclutador. Como se puede observar en la interfaz del sistema, el candidato puede rechazar horarios y el sistema propone alternativas automáticamente.



Imagen 15. Interfaz del agente de calendarización mostrando propuestas de horarios y opción de rechazo de Autoría propia (2025).

Prevención de duplicados en registro de comunicación

Un problema recurrente que enfrenté fue la creación de registros duplicados en la tabla `canal_comunicacion`. Para resolverlo, implementé verificaciones previas antes de cada inserción, especialmente en los agentes de feedback:

```
private function guardarLogComunicacion($conn, $idCliente, $mensaje, $canal, $estado = 'Enviado') {  
    // Verificar si ya existe un log similar para evitar duplicados  
    $sql_check = "SELECT id_comunicacion FROM canal_comunicacion  
                WHERE id_clientes = ? AND mensaje = ? AND canal = ? AND fecha = CURDATE()";  
    $stmt_check = $conn->prepare($sql_check);  
    $stmt_check->bind_param("iss", $idCliente, $mensaje, $canal);  
    $stmt_check->execute();  
    $result_check = $stmt_check->get_result();  
    $existe = $result_check->num_rows > 0;  
    $stmt_check->close();  
}
```

Imagen 16 fragmento de código donde se conecta al canal de comunicación Autoría propia (2025)

Esta estrategia de “verificar antes de insertar” se convirtió en un patrón que apliqué consistentemente en todos los agentes, mejorando significativamente la calidad de los datos almacenados.

9. Optimización del flujo de datos entre agentes

Una vez que todos los agentes individuales funcionaban correctamente, mi siguiente objetivo fue optimizar cómo interactuaban entre sí. Me di cuenta de que varios agentes necesitaban los mismos datos (por ejemplo, información del candidato y la vacante), pero cada uno estaba haciendo consultas independientes a la base de datos. Esto no era eficiente y podía causar problemas de rendimiento a medida que el sistema escalara.

Consolidación de lógica en el agente orquestador

Para el agente de segmentación, decidí mover toda la lógica directamente al método `ejecutarSegmentacion()` dentro del orquestador, en lugar de hacer un `include` del archivo `procesar_fit.php`. Esta decisión arquitectónica me permitió tener mayor control sobre el flujo y reducir la complejidad. El agente ahora obtiene los datos directamente con una sola consulta, calcula los scores técnico y blando, asigna el segmento (A, B, C), y guarda los resultados en la tabla evaluaciones.

Esta consolidación no solo mejoró el rendimiento al reducir las llamadas a archivos externos, sino que también hizo el código más mantenible y fácil de depurar. Fue una de esas refactorizaciones que, una vez completada, hace que te preguntes por qué no lo hiciste desde el principio. El resultado es un sistema más eficiente donde el agente de segmentación se ejecuta más rápido y los resultados se reflejan inmediatamente en el panel de evaluación.

Estrategia de ejecución en cascada para postulaciones

Uno de los flujos más críticos que optimicé fue el que se ejecuta cuando un candidato se postula a una vacante. Este proceso activa múltiples agentes en secuencia: primero el agente de segmentación calcula los scores y asigna el segmento, luego el agente de detección de riesgos analiza la aplicación usando esos scores, y finalmente se genera una alerta si el score de riesgo es alto. Esta cascada de ejecución garantiza que cada agente tenga los datos necesarios que fueron generados por agentes anteriores, creando un pipeline de procesamiento eficiente.

El resultado de este flujo se refleja inmediatamente en el sistema: cuando un candidato se postula, en cuestión de segundos el sistema ya tiene calculados sus scores, segmento y riesgos detectados, todo visible para el reclutador en el panel de evaluación. Fue muy satisfactorio ver cómo el sistema pasaba de simplemente “guardar una postulación” a “entender y evaluar el perfil” del candidato en tiempo real.

id_evaluacion	id_aplicacion	score_tecnico	score_blando	score_global	clasificacion_fit	segmento	comentarios_tecnicos	fecha_evaluacion
1	1	95	80	88	NULL	C	Excelente nivel de JS, le falta un poco de trabajo...	2025-11-24 11:11:11
2	17	0	50	25	Bajo Fit	C	Análisis Automático: Se detectaron coincidencias e...	2025-12-17 20:29:53
3	18	0	50	25	Bajo Fit	C	Análisis Automático: Se detectaron coincidencias e...	2025-12-17 20:51:31
4	19	60	50	55	Medio Fit	C	Análisis Automático: Se detectaron coincidencias e...	2025-12-17 21:00:14
5	20	100	50	75	Alto Fit	B	Análisis Automático: Se detectaron coincidencias e...	2025-12-17 21:02:39
6	21	0	50	25	Bajo Fit	C	Análisis Automático: Se detectaron coincidencias e...	2025-12-19 14:40:43
7	22	0	50	25	Bajo Fit	C	Análisis Automático: Se detectaron coincidencias e...	2025-12-27 15:32:38

Imagen 17 Panel de evaluación mostrando los resultados de la cascada de agentes tras una postulación de Autoría propia (2025).

Registro de logs unificado

Implementé un sistema de logging unificado en la tabla `log_agentes` que registra cada ejecución de un agente, incluyendo los datos de entrada y salida en formato JSON. Esta estructura me permitió crear una auditoría completa del sistema. Cada vez que un agente se ejecuta, se registra su inicio, estado, datos de entrada, datos de salida, y cualquier error que pueda ocurrir.

Estos logs fueron fundamentales durante el desarrollo para identificar dónde y por qué fallaban los agentes, y continúan siendo valiosos para el monitoreo del sistema en producción. El sistema de logs permite ver el historial completo de ejecuciones de cada agente, facilitando la depuración y el análisis del comportamiento del sistema. Como se puede observar en la interfaz de logs, se registran todos los agentes ejecutados junto con su estado y resultados.

id_log	id_aplicacion	agente_nombre	estado	datos_entrada	datos_salida	fecha_inicio	fecha_fin	error_mensaje
1	11	segmentacion	procesando	{"id_aplicacion":11}	NULL	2025-12-03 11:57:10	NULL	NULL
2	12	segmentacion	procesando	{"id_aplicacion":12}	NULL	2025-12-03 12:01:37	NULL	NULL
3	1	calendarizacion	completado	{"accion":"crear","id_aplicacion":"1","fecha_base":...	{"success":true,"id_entrevista":1,"opciones":...}	2025-12-03 12:30:14	2025-12-03 12:30:14	NULL
4	12	feedback_rechazo	procesando	{"id_aplicacion":"12","razones":["Score bajo","No ...	NULL	2025-12-03 12:30:38	NULL	NULL
5	12	feedback_rechazo	completado	{"id_aplicacion":"12","razones":["Score bajo","No ...	{"success":true,"mensaje":"Feedback generado y env...	2025-12-03 12:33:18	2025-12-03 12:33:19	NULL
6	1	calendarizacion	completado	{"accion":"crear","id_aplicacion":"1","fecha_base":...	{"success":true,"id_entrevista":1,"opciones":...}	2025-12-03 12:36:21	2025-12-03 12:36:21	NULL
7	12	validacion_proceso	procesando	{"id_aplicacion":"12"}	NULL	2025-12-03 17:05:31	NULL	NULL
8	11	validacion_proceso	procesando	{"id_aplicacion":"11"}	NULL	2025-12-03 17:05:33	NULL	NULL
9	12	validacion_proceso	procesando	{"id_aplicacion":"12"}	NULL	2025-12-03 17:05:38	NULL	NULL
10	2	validacion_proceso	procesando	{"id_aplicacion":"2"}	NULL	2025-12-03 17:08:43	NULL	NULL

Imagen 18 Interfaz de logs mostrando el registro unificado de ejecuciones de agentes de Autoría propia (2025).

10. Integración con sistema inicial y mantenimiento de compatibilidad

Una de las decisiones arquitectónicas más importantes que tomé fue mantener total compatibilidad con mi sistema existente que ya estaba funcionando. No podía permitirme romper funcionalidades que ya estaban siendo usadas por los reclutadores en producción, ni los agentes que ya tenía conectados. Esta restricción, aunque limitó algunas opciones de diseño, fue fundamental para asegurar que el sistema pudiera evolucionar sin interrupciones, integrando la nueva parte sin afectar lo que ya funcionaba.

Punto central de configuración y compatibilidad bidireccional

La integración de la nueva parte del sistema con mi sistema existente requirió una estrategia cuidadosa de compatibilidad bidireccional. Todos los agentes (tanto los que ya tenía como los nuevos) utilizan el mismo archivo config.php para obtener la conexión a la base de datos, pero este archivo también debe ser compatible con todo el código existente de mi sistema que ya estaba funcionando.

La solución que implementé fue crear una variable global \$mysqli para compatibilidad con mi código existente, y una función connection() para la nueva parte del sistema. Esta arquitectura me permitió mantener funcionando todo mi código existente sin modificaciones, mientras proporcionaba una API moderna para la nueva parte. También

centralicé la gestión de sesiones en un solo lugar, eliminando warnings de múltiples inicializaciones que podrían aparecer al integrar ambas partes.

Lo más importante es que ambas formas de acceso comparten la misma conexión, evitando problemas de transacciones o estados inconsistentes entre mi sistema existente y la nueva parte integrada. Esta centralización fue fundamental para que todo el sistema funcionara como una unidad cohesiva, donde los agentes nuevos y existentes pueden trabajar juntos sin conflictos.

Mapeo de esquemas y transformación de datos

Durante la adaptación de la nueva parte del sistema, una de las tareas más laboriosas fue mapear todas las diferencias entre los nombres de columnas esperados y los reales. La nueva parte del sistema esperaba ciertos nombres de columnas que no existían en mi estructura de base de datos recursosh.sql.

Para la tabla candidatos, el mapeo fue relativamente directo usando alias SQL. Para la tabla vacantes, el mapeo fue más complejo porque había múltiples diferencias que resolví usando COALESCE para flexibilidad. Para la tabla reclutadores, tuve que adaptar los nombres según el contexto de uso.

Estos mapeos no solo requerían cambios en las consultas SELECT, sino también en los INSERT y UPDATE. Por ejemplo, al crear una vacante desde el formulario de creación, el sistema mapea automáticamente los campos del formulario a los nombres correctos de columnas en mi base de datos. Esta estrategia de mapeo me permitió mantener la funcionalidad completa de la nueva parte del sistema sin necesidad de modificar la estructura de mi base de datos existente, que ya estaba siendo utilizada por mi sistema y los agentes que ya tenía funcionando. El resultado es que ambas partes del sistema trabajan perfectamente integradas, como se puede observar en el panel de administración donde se gestionan vacantes y candidatos sin problemas.

Candidatos.

Columna original	Alias usado	Uso
Id	idClientes	Foreign key en entrevistas, aplicaciones
Nombre	NombreCompleto	Display en interfaces
Email	Email	Directo
Teléfono	teléfono	Directo

Figura 2 Mapeo de tabla candidatos

Vacantes

Columna original	Mapeo	Uso
Id_vacante	idVacante	Identificador principal
Empresa	departamento	Identificador principal
Tipo_trabajo	departamento	Display como “Departamento”
Fecha_publicacion	fechaApertura/fechaCierre	Fecha de apertura/cierre

Figura 3 mapeo de tabla vacantes

Prevención de conflictos en múltiples ejecuciones

Para evitar que el mismo agente procese la misma aplicación múltiples veces simultáneamente, implementé verificaciones de estado en las consultas. Por ejemplo, el agente de feedback automatizado solo procesa aplicaciones que tienen el estado “Rechazado” y que no tienen un feedback previo. Este límite de 10 registros por ejecución también previene que un único proceso consuma demasiados recursos, permitiendo que el sistema procese los rechazados en lotes manejables. El resultado es un sistema más estable y eficiente, como se puede observar en los logs del sistema donde se registra el número de procesados en cada ejecución automática.

CAPÍTULO 3.

PROPUESTAS Y

RESULTADOS

1. Resultados obtenidos

Tras la implementación y despliegue del sistema de orquestación de agentes, se alcanzaron logros importantes tanto en el rendimiento técnico como en la funcionalidad operativa del Agente de Segmentación de Audiencia.

Optimización del tiempo de procesamiento

Uno de los resultados mas destacados es la disminución significativa en los tiempos de respuesta. Mientras que un reclutador humano puede tardar varios minutos en analizar detalladamente un CV, el agente logra procesar y segmentar la información en un tiempo máximo de 5 segundos. Este rendimiento se basa en la lógica comparación contra un perfil de “candidato perfecto” predefinido, donde el sistema busca una coincidencia mínima del 80% en habilidades y experiencia para clasificar al candidato.

Precisión y validación del modelo

Durante las pruebas de estrés y validación, el sistema demostró una alta fidelidad en sus diagnósticos:

- Prueba de coincidencia: Al procesar un CV con habilidades técnicas alineadas a la vacante, el sistema arrojó un Fit Score del 80%, validando la efectividad de los embeddings y las instrucciones del agente.
- Detección de riesgos Al introducir archivos con información irrelevante o carente de habilidades profesionales, el sistema asignó una calificación mínima y activó automáticamente el Agente de Detección de Riesgos, cumpliendo con los protocolos de seguridad y filtrado establecidos.

Estabilidad del backend e integración

A pesar de ser el único responsable del backend tras la deserción del equipo inicial, logré consolidar una arquitectura sólida:

- Integridad de datos Se resolvieron satisfactoriamente los problemas de concurrencia y duplicidad en MySQL, garantizando que cada registro sea único y coherente.
- Comunicación en tiempo real: Mediante la integración de la lógica en las vistas del frontend y la conexión con la base de datos, se logró que todas las acciones de los

agentes (mensajes y resultados) se reflejaran instantáneamente en la interfaz del usuario, permitiendo un monitoreo transparente de la "vida propia" que adquiere la IA durante el proceso.

2. Mejoras a futuro y propuestas

El sistema actual sienta las bases para una herramienta de reclutamiento autónoma de nivel empresarial; sin embargo, se proponen las siguientes líneas de evolución:

- Migración tecnológica, Aunque PHP permitió una implementación ágil y familiar, se recomienda la migración del motor de IA hacia Python con FastAPI. Esto facilitaría el uso de librerías nativas de aprendizaje profundo y optimizaría la gestión de modelos de lenguaje más pesados.
- Unificación de módulos Actualmente, existen componentes del sistema que operan de forma aislada, aunque compartan la misma base de datos. Se propone una integración total en una plataforma "todo en uno" para mejorar la experiencia del administrador.
- Agente de Entrevistas Virtuales: Implementar un módulo de videollamada donde un agente de IA pueda realizar una entrevista preliminar, analizando el lenguaje y las respuestas del candidato en tiempo real para complementar el criterio del reclutador humano.
- Agente Diseñador Creativo: Desarrollar un agente especializado en la creación de activos visuales (imágenes y banners) para la publicación de vacantes en redes sociales, automatizando el marketing de reclutamiento.

CONCLUSIÓN, RECOMENDACIÓN y EXPERIENCIA PERSONAL PROFESIONAL ADQUIRIDA

El proyecto de residencia profesional cumplió con el objetivo general y superó las expectativas iniciales. Lo que comenzó como la creación de un único agente de segmentación evolucionó hacia el desarrollo y orquestación de un sistema backend integral.

Se logró demostrar que la inteligencia artificial, cuando está correctamente orquestada mediante instrucciones precisas y herramientas como Gemini CLI, deja de ser una herramienta pasiva para convertirse en un ente con capacidad operativa que reduce la carga administrativa en más de un 90%. La culminación de este sistema no solo automatiza tareas, sino que dota a la empresa de una infraestructura escalable para el reclutamiento inteligente.

A la empresa Develop Talent & Technology se le sugiere mantener una supervisión técnica rigurosa sobre los proyectos de residencia, fomentando un entorno de "presión real" similar al mercado laboral actual. Esto preparará de mejor manera a los futuros ingenieros ante situaciones de crisis, como la falta de respuesta de colaboradores o plazos de entrega ajustados. Asimismo, se recomienda la actualización periódica de los prompts de sistema de los agentes para evitar el "sesgo de deriva" en las evaluaciones de los candidatos.

Mi paso por esta residencia ha sido la etapa más transformadora de mi formación académica. El mayor desafío no fue escribir código, sino lograr la orquestación de agentes; pasar de instrucciones estáticas a lograr que la IA "pensara" y actuara bajo flujos lógicos complejos fue un muro técnico que requirió profunda investigación y experimentación.

A nivel profesional, el asumir la responsabilidad total del backend ante la ausencia de mi equipo me brindó una autoconfianza invaluable. Aprendí que la integridad de un sistema depende de la conciencia total sobre cada línea de código escrita. Hoy me retiro con una especialización práctica en la creación de ecosistemas de IA, comprendiendo no solo cómo programarlos, sino cómo "razona" la inteligencia artificial para resolver problemas del mundo real.

COMPETENCIAS	DESARROLLADAS	Y/O
APLICADAS		

1. Programación orientada a objetos. Competencia de la asignatura: "Aplica el paradigma orientado a objetos para el desarrollo de aplicaciones que solucionen problemas del entorno."
2. Taller de base de datos. Competencia de la asignatura: "Crea y aplica esquemas de bases de datos para garantizar la confiabilidad de los datos en aplicaciones para el tratamiento de información."
3. Programación Web. Competencia de la asignatura: "Desarrolla aplicaciones Web dinámicas con la finalidad de ofrecer soluciones informáticas a las organizaciones, utilizando programación del lado del cliente y del lado del servidor."
4. Ingeniería de software. Competencia de la asignatura: "Conoce y aplica conceptos, métodos y/o técnicas para el desarrollo de software de calidad, administrando proyectos que resuelvan problemas del entorno."
5. Ingeniería del conocimiento. Competencia de la asignatura: " Identifica técnicas de modelado y representación de conocimiento útil para la toma de decisiones."

REFERENCIAS BIBLIOGRÁFICAS

Deel. (2025). Qué es y por qué importa el reclutamiento de personal. Recuperado de <https://www.deel.com/es/blog/reclutamiento-de-personal/>

IBM. (2025). Agentes de IA en recursos humanos. Recuperado de <https://www.ibm.com/es-es/think/topics/ai-agents-in-human-resources>

IBM. (2025). ¿Qué es el sesgo algorítmico? Recuperado de <https://www.ibm.com/mx-es/think/topics/algorithmic-bias>

Insight7. (2025). Unsupervised Learning Evaluation Metrics Explained. Recuperado de <https://insight7.io/unsupervised-learning-evaluation-metrics-explained/>

iSmartRecruit. (2025). Reclutamiento y Selección: La Función de RRHH más Crítica. Recuperado de <https://www.ismartrecruit.com/es/blogs/reclutamiento-y-seleccion-funcion-critica-rrhh>

Milvus. (2025). How do Sentence Transformers differ from traditional word embedding models like Word2Vec or GloVe? Recuperado de <https://milvus.io/ai-quick-reference/how-do-sentence-transformers-differ-from-traditional-word-embedding-models-like-word2vec-or-glove>

New Horizons. (2024). DBSCAN vs. K-Means: A Guide in Python. Recuperado de <https://www.newhorizons.com/resources/blog/dbscan-vs-kmeans-a-guide-in-python>

OpenWebinars. (2024). Por qué usar un ATS en el proceso de reclutamiento. Recuperado de <https://openwebinars.net/blog/por-que-usar-un-ats-en-el-proceso-de-reclutamiento/>

ServiceNow. (s.f.). ¿Qué son los agentes de IA? Recuperado de <https://www.servicenow.com/latam/products/ai-agents/what-are-ai-agents.html>

Automation Anywhere. (2025). ¿Qué es la automatización de procesos con agentes?. Recuperado de <https://www.automationanywhere.com/la/rpa/agentic-process-automation>

Factorial. (2025). Reclutamiento y selección de personal de una empresa, ¿es lo mismo?. Recuperado de <https://factorial.mx/blog/reclutamiento-seleccion-personal/>

Fortinet. (2025). ¿Qué es el análisis heurístico? Métodos de detección y eliminación. Recuperado de <https://www.fortinet.com/lat/resources/cyberglossary/heuristic-analysis>

Genomawork. (2022). ¿Qué son las heurísticas y cómo se cuelean en tus decisiones?. Recuperado de <https://www.genoma.work/post/que-es-heuristica-como-afecta-decision-de-contratar-personal>

Google Cloud. (2025). ¿Qué es un sistema multiagente en la IA?. Recuperado de <https://cloud.google.com/discover/what-is-a-multi-agent-system?hl=es-419>

IBM. (2025). ¿Qué es la orquestación de microservicios?. Recuperado de <https://www.ibm.com/mx-es/think/topics/microservices-orchestration>

Urbe. (2009). Capítulo II: Definición de Reclutamiento. Recuperado de <https://virtual.urbe.edu/tesispub/0097271/cap02.pdf>

Xygeni. (2023). ¿Qué es la orquestación en el desarrollo de software?. Recuperado de <https://xygeni.io/es/sscs-glossary/what-is-orchestration-in-software-development/>